

Отчёт по модулю сети Фейстеля

Филатов, Сергеева, КБ-221

Описание всей программы

Модуль `feistel.py` реализует сеть Фейстеля для шифрования и дешифрования строк в кодировке UTF-8. Процесс разбивается на подготовку исходного текста к работе с 16-битными блоками, проведение восьми раундов преобразований в каждом блоке с использованием циклических сдвигов ключа, а также обратное преобразование шифртекста в исходную строку. Дополнительно модуль ведёт подробные текстовые журналы на каждом этапе, чтобы демонстрировать внутренние состояния алгоритма: формирование блоков, последовательные раунды сети, применение XOR между блоками и восстановление открытого текста. Вспомогательные функции обеспечивают проверку корректности ключа, форматирование блоков, операции XOR и вращения, а структура `PreparedText` хранит результаты подготовки текста.

В проекте также присутствует веб-интерфейс на Flask (файл `app.py`), позволяющий взаимодействовать с реализацией сети через браузер (постоянная ссылка <http://45.156.20.75:2141>).

Подробное описание функций модуля `feistel.py`

`PreparedText` (dataclass)

```
@dataclass
class PreparedText:
    """Результат подготовки открытого текста."""

    binary_blocks: List[str]
    original_byte_length: int
```

`PreparedText` — простая структура данных, которая фиксирует результат предварительной обработки текста.

Поле `binary_blocks` хранит 16-битные двоичные строки, готовые к подаче в раунды сети Фейстеля, а `original_byte_length` позволяет при восстановлении открытого текста отбросить добавленный выравнивающий байт.

`_bytes_to_blocks`

```
def _bytes_to_blocks(data: bytes) -> List[str]:

    blocks: List[str] = []
    for index in range(0, len(data), 2):
        combined = (data[index] << 8) | data[index + 1]
        blocks.append(format(combined, "016b"))
    return blocks
```

Функция принимает массив байтов и объединяет их попарно в 16-битные целые числа. Каждый шаг цикла берёт два последовательных байта, сдвигает старший на восемь бит влево и объединяет с младшим через операцию **XOR**. Полученное значение форматируется как двоичная строка фиксированной длины 16 символов и добавляется в список. Эта подготовка необходима для обработки текста блоками в сети Фейстеля.

`_blocks_to_bytes`

```
def _blocks_to_bytes(blocks: Sequence[str]) -> bytearray:

    result = bytearray()
    for block in blocks:
        value = int(block, 2)
        result.append((value >> 8) & 0xFF)
        result.append(value & 0xFF)
    return result
```

Обратная функция для `_bytes_to_blocks`: она принимает двоичные строки, преобразует каждую в целое число, затем выделяет старший и младший байты с помощью сдвигов и маскирования, добавляя их в результирующий **bytearray**. Это позволяет получить последовательность байтов, эквивалентную исходной строке (с учётом возможного добавленного байта выравнивания).

`prepare_plaintext_with_trace`

```
def prepare_plaintext_with_trace(plaintext: str) -> Tuple[PreparedText, str]:
    """Преобразует текст в блоки и возвращает журнал операций."""

    lines: List[str] = []
    lines.append(f"Исходный текст: {plaintext!r}")

    encoded = plaintext.encode(ENCODING)
    grouped_bytes = " ".join(format(byte, "08b") for byte in encoded) or "-"
    lines.append(f"Текст в UTF-8 (байты): {grouped_bytes}")
    lines.append(f"Длина: {len(encoded)} байт")

    data = bytearray(encoded)
    if len(data) % 2 != 0:
        data.append(PADDING_BYTE)
        lines.append(
            "Дополнено до чётного количества байт: добавлен байт "
            f"0x{PADDING_BYTE:02X}"
        )
    lines.append(f"Длина после дополнения: {len(data)} байт")

    blocks = _bytes_to_blocks(bytes(data))
    lines.append(f"Количество блоков: {len(blocks)}")
    for index, block in enumerate(blocks, start=1):
```

```

        byte_pair = data[(index - 1) * 2 : index * 2]
        lines.append(
            f"Блок {index}: {block} (байты {format(byte_pair[0], '08b')} "
            f"{format(byte_pair[1], '08b')})"
        )

    prepared = PreparedText(binary_blocks=blocks,
                             original_byte_length=len(encoded))
    return prepared, "\n".join(lines)

```

Функция готовит произвольную строку к шифрованию: кодирует текст в UTF-8, выводит байты в виде двоичных строк и при необходимости дополняет массив байтом нулей для получения чётного количества байтов. После этого она разбивает данные на 16-битные блоки, ведёт подробный лог с описанием каждого блока и возвращает структуру `PreparedText` вместе с текстовым журналом. Журнал удобен для визуального анализа предварительного этапа алгоритма.

prepare_plaintext

```

def prepare_plaintext(plaintext: str) -> PreparedText:
    """Совместимая обёртка без журнала."""

    prepared, _ = prepare_plaintext_with_trace(plaintext)
    return prepared

```

Обёртка над `prepare_plaintext_with_trace`, скрывающая текстовый журнал. Она возвращает только структуру `PreparedText` и подходит для сценариев, где не нужен подробный вывод.

restore_plaintext_with_trace

```

def restore_plaintext_with_trace(blocks: Sequence[str], original_byte_length: int)
-> Tuple[str, str]:
    """Преобразует блоки обратно в текст и возвращает журнал операций."""

    lines: List[str] = []
    lines.append(f"Количество блоков на входе: {len(blocks)}")

    byte_array = _blocks_to_bytes(blocks)
    grouped_bytes = " ".join(format(byte, "08b") for byte in byte_array)
    lines.append(f"Блоки в байтах (до обрезки): {grouped_bytes}")
    trimmed_bytes = bytes(byte_array[:original_byte_length])
    lines.append(f"Обрезано до {original_byte_length} байт исходного текста")
    lines.append(
        "Байты после обрезки: "
        + (" ".join(format(byte, "08b") for byte in trimmed_bytes) or "-")
    )

    plaintext = trimmed_bytes.decode(ENCODING)

```

```
lines.append(f"Восстановленный текст: {plaintext!r}")
return plaintext, "\n".join(lines)
```

Функция объединяет двоичные блоки в последовательность байтов, отрезает добавленный ранее байт выравнивания, декодирует результат в UTF-8 и формирует подробный журнал восстановления. Лог фиксирует количество блоков, промежуточные байтовые представления и конечный текст.

restore_plaintext

```
def restore_plaintext(blocks: Sequence[str], original_byte_length: int) -> str:
    """Обратное преобразование без журнала."""

    plaintext, _ = restore_plaintext_with_trace(blocks, original_byte_length)
    return plaintext
```

Обёртка без журнала для восстановления исходной строки. Используется, когда необходим только итоговый текст без сопроводительного вывода.

left_rotate_bits

```
def left_rotate_bits(value: str, shift: int) -> str:
    """Циклически сдвигает двоичную строку влево."""

    shift %= len(value)
    return value[shift:] + value[:shift]
```

Реализует циклический сдвиг битовой строки влево. Нормализует величину сдвига по длине строки, после чего формирует результат с помощью конкатенации хвоста и головы строки. Используется для получения ключей раундов из базового 16-битного ключа.

round_function

```
def round_function(right_half: str, round_key: str) -> str:
    """Раундовая функция F: XOR + сложение по модулю 256."""

    left_key = round_key[:8]
    right_key = round_key[8:]

    right_value = int(right_half, 2)
    left_key_value = int(left_key, 2)
    right_key_value = int(right_key, 2)

    mixed = right_value ^ left_key_value
    transformed = (mixed + right_key_value) % 256
    return format(transformed, "08b")
```

Раундовая функция **F** принимает правую половину блока и 16-битный раундовый ключ. Ключ разбивается на две части: старшие 8 бит участвуют в XOR с правой половиной, а младшие 8 бит прибавляются по модулю 256 к результату XOR. Возвращается новая 8-битная строка, используемая при вычислении следующего состояния блока.

feistel_rounds

```
def feistel_rounds(block: str, key: str) -> str:
    """Шифрует один блок без трассировки."""

    encrypted, _ = feistel_rounds_with_trace(block, key)
    return encrypted
```

Обёртка, выполняющая шифрование одного 16-битного блока без построения журнала. Возвращает только зашифрованную строку, используя реализацию с трассировкой под капотом.

feistel_rounds_with_trace

```
def feistel_rounds_with_trace(block: str, key: str) -> Tuple[str, List[str]]:
    """Шифрует блок и возвращает журнал по раундам."""

    lines: List[str] = []
    left = block[:8]
    right = block[8:]
    lines.append(f"Начальное состояние: L={left}, R={right}")

    for round_number in range(1, 9):
        round_key = left_rotate_bits(key, round_number)
        f_output = round_function(right, round_key)
        xor_result_value = int(left, 2) ^ int(f_output, 2)
        xor_result = format(xor_result_value, "08b")
        lines.append(f"--- РАУНД {round_number} ---")
        lines.append(f"Ключ раунда: {round_key}")
        lines.append(f"F(R, K) = F({right}, {round_key}) = {f_output}")
        lines.append(f"L ⊕ F(R,K) = {left} ⊕ {f_output} = {xor_result}")
        new_left = right
        new_right = xor_result
        lines.append(f"Новое состояние: L={new_left}, R={new_right}")
        left, right = new_left, new_right

    lines.append(f"Финальное состояние: L={left}, R={right}")
    result = left + right
    lines.append(f"Зашифрованный блок: {result}")
    return result, lines
```

Основная реализация восьми раундов сети Фейстеля. На каждом шаге выполняется циклический сдвиг ключа, вычисление функции **F**, XOR с левой половиной и обмен половин. В журнал записываются все

промежуточные состояния, что позволяет проанализировать поток данных раунда за раундом. После завершения раундов части склеиваются в финальный шифроблок.

feistel_inverse_rounds

```
def feistel_inverse_rounds(block: str, key: str) -> str:
    """Расшифровывает блок без трассировки."""

    decrypted, _ = feistel_inverse_rounds_with_trace(block, key)
    return decrypted
```

Обратная обёртка без журнала, выполняющая расшифрование одного блока. Как и в случае шифрования, использует версию с трассировкой, но скрывает подробный лог.

feistel_inverse_rounds_with_trace

```
def feistel_inverse_rounds_with_trace(block: str, key: str) -> Tuple[str,
List[str]]:
    """Расшифровывает блок и возвращает журнал по раундам."""

    lines: List[str] = []
    left = block[:8]
    right = block[8:]
    lines.append(f"Начальное состояние: L={left}, R={right}")

    for round_number in range(8, 0, -1):
        round_key = left_rotate_bits(key, round_number)
        f_output = round_function(left, round_key)
        xor_result_value = int(right, 2) ^ int(f_output, 2)
        xor_result = format(xor_result_value, "08b")
        lines.append(f"--- РАУНД {round_number} ---")
        lines.append(f"Ключ раунда: {round_key}")
        lines.append(f"F(L, K) = F({left}, {round_key}) = {f_output}")
        lines.append(f" $R \oplus F(L, K) = \{right\} \oplus \{f\_output\} = \{xor\_result\}$ ")
        new_left = xor_result
        new_right = left
        lines.append(f"Новое состояние: L={new_left}, R={new_right}")
        left, right = new_left, new_right

    lines.append(f"Финальное состояние: L={left}, R={right}")
    result = left + right
    lines.append(f"Расшифрованный блок: {result}")
    return result, lines
```

Производит обратный проход по раундам сети, начиная с восьмого и заканчивая первым. На каждом шаге переиспользуется `round_function`, но применяется к левой половине, чтобы восстановить оригинальные значения. Журнал позволяет увидеть, как происходят обратные преобразования и обмен половин.

xor_blocks

```
def xor_blocks(first: str, second: str) -> str:
    """Выполняет XOR двух 16-битных строк."""

    value = int(first, 2) ^ int(second, 2)
    return format(value, "016b")
```

Функция побитово складывает по модулю два две 16-битные строки. Применяется как при шифровании (для реализации сцепления блоков), так и при расшифровании.

encrypt_blocks_with_trace

```
def encrypt_blocks_with_trace(blocks: Sequence[str], key: str) -> Tuple[List[str],
str]:
    """Шифрует блоки и возвращает подробный журнал процесса."""

    lines: List[str] = []
    lines.append("=====")
    lines.append("ШИФРОВАНИЕ")
    lines.append("=====")
    lines.append(f"Исходный ключ: {key}")

    encrypted: List[str] = []
    previous_cipher = "0" * 16

    for index, block in enumerate(blocks, start=1):
        lines.append("")
        lines.append(f"ШИФРОВАНИЕ БЛОКА {index}: {block}")
        if index > 1:
            xored = xor_blocks(block, previous_cipher)
            lines.append(
                f"XOR с предыдущим шифроблоком: {block}  $\oplus$  {previous_cipher} =
{xored}"
            )
            block_to_encrypt = xored
        else:
            lines.append("Первый блок шифруется без предварительного XOR.")
            block_to_encrypt = block

        encrypted_block, trace = feistel_rounds_with_trace(block_to_encrypt, key)
        lines.extend(trace)
        encrypted.append(encrypted_block)
        previous_cipher = encrypted_block

    lines.append("")
    lines.append("ВСЕ БЛОКИ ЗАШИФРОВАНЫ")
    lines.append(
        "Полный зашифрованный текст: " + "".join(encrypted)
```

```
)
return encrypted, "\n".join(lines)
```

Реализует шифрование последовательности блоков с формированием детального журнала. Для блоков, кроме первого, выполняется XOR с предыдущим шифроблоком (аналог режима CBC), что отражается в логе. Затем каждый блок проходит восемь раундов сети Фейстеля, а результаты и промежуточные состояния включаются в журнал. В конце возвращается список зашифрованных блоков и полный текст отчёта.

encrypt_blocks

```
def encrypt_blocks(blocks: Sequence[str], key: str) -> List[str]:
    """Совместимая версия без журнала."""

    encrypted, _ = encrypt_blocks_with_trace(blocks, key)
    return encrypted
```

Упрощённая версия пакетного шифрования, возвращающая только список шифроблоков без текстового описания процесса.

decrypt_blocks_with_trace

```
def decrypt_blocks_with_trace(blocks: Sequence[str], key: str) -> Tuple[List[str],
str]:
    """Расшифровывает блоки и возвращает подробный журнал процесса."""

    lines: List[str] = []
    lines.append("=====")
    lines.append("ДЕШИФРОВАНИЕ")
    lines.append("=====")
    lines.append(f"Исходный ключ: {key}")

    decrypted: List[str] = []
    previous_cipher = "0" * 16

    for index, block in enumerate(blocks, start=1):
        lines.append("")
        lines.append(f"ДЕШИФРОВАНИЕ БЛОКА {index}: {block}")
        decrypted_block, trace = feistel_inverse_rounds_with_trace(block, key)
        lines.extend(trace)

        if index > 1:
            xored = xor_blocks(decrypted_block, previous_cipher)
            lines.append(
                f"Обратный XOR с предыдущим шифроблоком: {decrypted_block} ⊕ "
                f"{previous_cipher} = {xored}"
            )
            decrypted_block = xored
```



```

        else:
            lines.append("Первый блок не требует обратного XOR.")

            decrypted.append(decrypted_block)
            previous_cipher = block

    lines.append("")
    lines.append("ВСЕ БЛОКИ ДЕШИФРОВАНЫ")
    lines.append(
        "Полный дешифрованный текст: " + "".join(decrypted)
    )
    return decrypted, "\n".join(lines)

```

Пошагово расшифровывает каждый блок, используя обратные раунды сети Фейстеля и журналируя все промежуточные вычисления. После каждого блока, начиная со второго, выполняется XOR с предыдущим шифроблоком для отмены сцепления. Итоговый журнал подробно описывает весь процесс восстановления.

decrypt_blocks

```

def decrypt_blocks(blocks: Sequence[str], key: str) -> List[str]:
    """Совместимая версия без журнала."""

    decrypted, _ = decrypt_blocks_with_trace(blocks, key)
    return decrypted

```

Функция возвращает только последовательность расшифрованных 16-битных блоков, скрывая текстовый отчёт.

ensure_16bit_key

```

def ensure_16bit_key(key: str) -> str:
    """Проверяет, что ключ состоит из 16 бит."""

    if not set(key) <= {"0", "1"}:
        raise ValueError("Ключ должен состоять только из символов '0' и '1'.")
    if len(key) != 16:
        raise ValueError("Ключ сети Фейстеля обязан быть длиной ровно 16 бит.")
    return key

```

Валидирует двоичный ключ: убеждается, что он содержит только символы **0** и **1**, и что его длина равна 16. При нарушении условий возбуждает **ValueError**. Возвращает исходную строку для удобства цепочек вызовов.

format_blocks

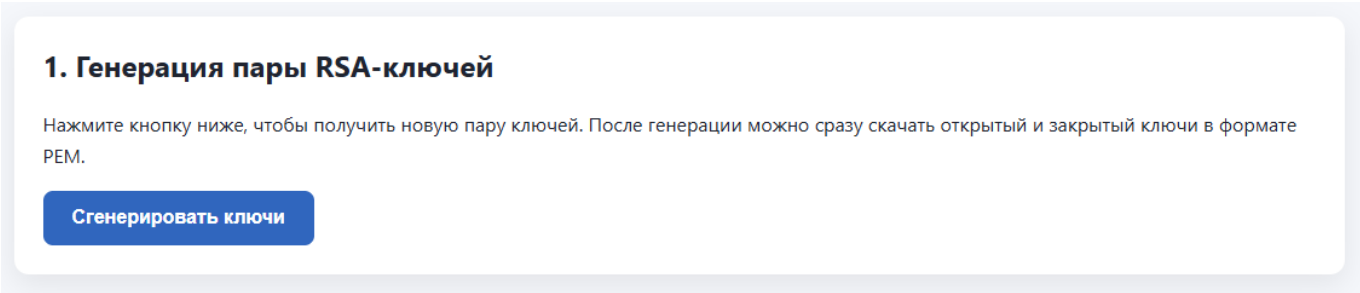
```
def format_blocks(blocks: Sequence[str]) -> str:
    """Возвращает текстовое представление блоков."""

    return " ".join(blocks)
```

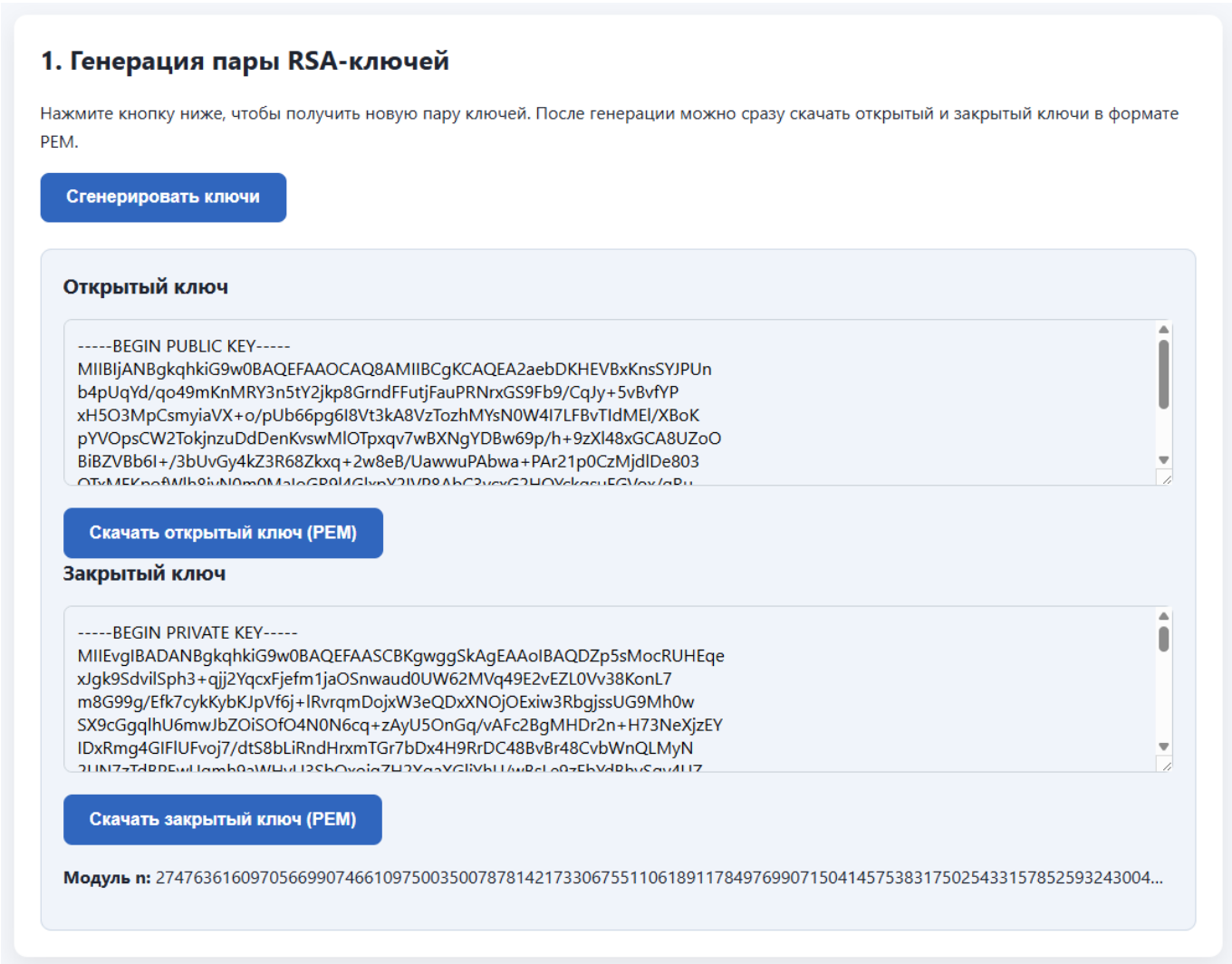
Удобная функция для форматирования списка блоков в строку, где отдельные блоки разделены пробелами. Используется для вывода пользователю.

Скриншоты

Интерфейс генерации RSA ключей



Сгенерированные RSA ключи



Интерфейс шифрования сообщения

2. Шифрование сообщения

Введите текст и предоставьте открытый ключ получателя. Симметричный 16-битный ключ генерируется автоматически и шифруется RSA.

Открытый текст

Файл открытого ключа (PEM)

Выберите файл

Файл не выбран

Либо вставьте открытый ключ вручную

Зашифровать

Пример шифрования сообщения

2. Шифрование сообщения

Введите текст и предоставьте открытый ключ получателя. Симметричный 16-битный ключ генерируется автоматически и шифруется RSA.

Открытый текст

test1 тест2

Файл открытого ключа (PEM)

Выберите файл

rsa_public_key.pem

Либо вставьте открытый ключ вручную

Зашифровать

Результаты шифрования

Результаты шифрования

СИММЕТРИЧНЫЙ КЛЮЧ (16 БИТ)
0010111111010111

ЗАШИФРОВАННЫЙ КЛЮЧ RSA
(BASE64)
MC15vM6VvRZYh/B15ohxIyv7hBtM
NBmtaLxQQWR9uuo8/S0puhdbKtgJ
wiiJ75p6B4qqvkfHfDVgnGud/+iJ
ZNvHxp+oc0nNF9vWJaBrFdtSU02j
W09LFMyZ3etpNovE12iJExr09Ask
rR1TYRi40rvw1QGyDF0+MnTuORD1
5xZr3R20aQySrIKovqM/uKwGIrh6
sA3B47uPXa7PkRgdtmdQtTXXDdb
LYRr/spFuKmuZgPPhGBvnVKmTZDR
KUwepYwL44Z5svzF/IjzQck3adIa
h06zRi+PPGZqOvJyvWG5Ban1lRD
YR15f1EWmj5QaaBbAp4bLDFV1TqG
GcXytg==

РАЗМЕР ИСХОДНОГО ТЕКСТА
15 байт

Блоки шифртекста

Всего 8

- 1001011011010110
- 1010111001110001
- 1110100100001101
- 1110100010010010
- 0001000000010010
- 0110110101011011
- 1111001011001110
- 1011011110000000

Результаты шифрования

СИММЕТРИЧНЫЙ КЛЮЧ (16 БИТ)
0010111111010111

ЗАШИФРОВАННЫЙ КЛЮЧ RSA
(BASE64)
MC15vM6VvRZYh/B15ohxIyv7hBtM
NBmtaLxQQWR9uuo8/S0puhdbKtgJ
wiiJ75p6B4qqvkfHfDVgnGud/+iJ
ZNvHxp+oc0nNF9vWJaBrFdtSU02j
W09LFMyZ3etpNovE12iJExr09Ask
rR1TYRi40rvw1QGyDF0+MnTuORD1
5xZr3R20aQySrIKovqM/uKwGIrh6
sA3B47uPXa7PkRgdtmdQtTXXDdb
LYRr/spFuKmuZgPPhGBvnVKmTZDR
KUwepYwL44Z5svzF/IjzQck3adIa
h06zRi+PPGZqOvJyvWG5Ban1lRD
YR15f1EWmj5QaaBbAp4bLDFV1TqG
GcXytg==

РАЗМЕР ИСХОДНОГО ТЕКСТА
15 байт

Блоки шифртекста

Всего 8

- 1001011011010110
- 1010111001110001
- 1110100100001101
- 1110100010010010
- 0001000000010010
- 0110110101011011
- 1111001011001110
- 1011011110000000

ШИФРОВАНИЕ БЛОКА 1:**011010001100101**

- Первый блок шифруется без предварительного XOR.
- Начальное состояние:
L=01110100, R=01100101
- --- РАУНД 1 ---
- Ключ раунда:
0101111110101110
- $F(R, K) = F(01100101, 0101111110101110) = 11101000$
- $L \oplus F(R, K) = 01110100 \oplus 11101000 = 10011100$
- Новое состояние:
L=01100101, R=10011100
- --- РАУНД 2 ---
- Ключ раунда:
10111111101011100
- $F(R, K) = F(10011100, 10111111101011100) = 01111111$
- $L \oplus F(R, K) = 01100101 \oplus 01111111 = 00011010$
- Новое состояние:
L=10011100, R=00011010
- --- РАУНД 3 ---
- Ключ раунда:
01111111010111001
- $F(R, K) = F(00011010, 01111111010111001) = 00011101$
- $L \oplus F(R, K) = 10011100 \oplus 00011101 = 10000001$
- Новое состояние:
L=00011010, R=10000001

Зашифрованный пакет в формате json

```
{
  "version": 1,
  "cipher_blocks": [
    "1001011011010110",
    "1010111001110001",
    "1110100100001101",
    "1110100010010010",
    "0001000000010010",
    "0110110101011011",
    "1111001011001110",
    "1011011110000000"
  ],
  "encrypted_key": "MCL5vM6VvRZYh/BL5ohxIyv7hBtMNBmtaLxQQWR9uuo8/S0puhdbKtgJwiiJ75p6B4ggvkfHfDVgn6Ud/+iJZNvHxp",
  "rsa_modulus": "27476361609705669907466109750035007878142173306755110618911784976990715041457538317502543315",
  "original_byte_length": 15,
  "metadata": {
    "encoding": "utf-8",
    "padding_byte": "0x00",
    "feedback_mode": "XOR с предыдущим шифроблоком",
    "rounds": "8",
    "rsa": "RSA-OAEP (SHA-256)"
  }
}
```

Интерфейс расшифрования пакета

3. Расшифрование пакета

Получатель выбирает пакет и закрытый ключ, после чего система восстанавливает исходный текст и симметричный ключ.

Файл пакета (JSON)

Выберите файл

Файл не выбран

Файл закрытого ключа (PEM)

Выберите файл

Файл не выбран

Расшифровать

Расшифрование пакета

3. Расшифрование пакета

Получатель выбирает пакет и закрытый ключ, после чего система восстанавливает исходный текст и симметричный ключ.

Файл пакета (JSON)

Выберите файл

feistel_package (1).json

Файл закрытого ключа (PEM)

Выберите файл

rsa_private_key.pem

Расшифровать

Расшифрованные данные

СИММЕТРИЧНЫЙ КЛЮЧ (16 БИТ)
0110001100010100

КОЛИЧЕСТВО БЛОКОВ
8

Полученные шифроблоки

XOR режим обратной связи

- 1001111111001110
- 1100011000001100
- 0110100111001111
- 1101001100001010
- 0110100000101101
- 1101101111101101
- 00111111111000100
- 1111011101100001

Исходный текст

test1 тест2

Ход расшифрования

ДЕШИФРОВАНИЕ

- Исходный ключ:
0110001100010100

ДЕШИФРОВАНИЕ БЛОКА 1:
100111111001110

ДЕШИФРОВАНИЕ БЛОКА 2:
1100011000001100

ДЕШИФРОВАНИЕ БЛОКА 3:
010100111001111

ДЕШИФРОВАНИЕ БЛОКА 4:
1101001100001010

ДЕШИФРОВАНИЕ БЛОКА 5:
010100000101101

ДЕШИФРОВАНИЕ БЛОКА 6:
1101101111101101

ДЕШИФРОВАНИЕ БЛОКА 7:
0011111111000100

ДЕШИФРОВАНИЕ БЛОКА 8:
1111011101100001

ВСЕ БЛОКИ ДЕШИФРОВАНЫ

Восстановление текста

- Количество блоков на входе: 8
- Блоки в байтах (до обрезки):
01110100 01100101 01110011
01110100 00110001 00100000
11010001 10000010 11010000
10110101 11010001 10000001
11010001 10000010 00110010
00000000
- Обрезано до 15 байт исходного текста
- Байты после обрезки: 01110100
01100101 01110011 01110100
00110001 00100000 11010001
10000010 11010000 10110101
11010001 10000001 11010001
10000010 00110010
- Восстановленный текст: 'test1
тест2'

Шаги RSA

- RSA-РАСШИФРОВАНИЕ
СИММЕТРИЧНОГО КЛЮЧА
- Модуль n:
27630365604848260491723151363
59990906320155565867108769840
01955997965919898763044681387
34617517282011975874398977628
28983560084679450812802926268
46719468182320218412189555712
12075162251089525677462119006
13420677256566446485287986267
37586505267214638473298073527
97196245301046615750503251695
25372812082067161737017700862
82726917914927175588400179173
63041190999763923358543815607
08518431993687627871299371034
74571846726122210997525969931
21130409045330787018560572805
34158971231349866651260769971
49549775111190924487282615042
67434710764390290498862632677
06376205139110294098162185886
61850851173492837303936772088
14429783
- Входные данные (base64):
WwLyuqtHrdU8f5w8wAqPwMG7kgQTD
Q9gJjDZ2/67+CMAak5KV8HehmACUE
n5lXRNM/NaZlW1FLOWPjvu3Ed7eis
b6RNwIIFKb2rhTONax7K8xmbHtjGf
urMyLFSwQ2FbQI+KTpOJDyoIyuds8
qU8llsGT2pPKW00ZggAfHimCAYYgP
5665HP0mttsiK/X+Y2Cqe5q8XRQ27
6WYN878XAG4vrThUfZsoAJLHZDyca
kVgnzTRnHpKnre/2WwckN/qxkAm+Y
WG48Y/SGfwhm9fg8DztVnx40S8p3Y
9rw7bUTFCDsFp2p5NTY1r87clZOMz
bv08jsoIZEHOcGWU5m5Y5GA==
- Используется схема OAEP с
SHA-256.
- Результат в hex: 0x6314
- Результат в двоичном виде:
0110001100010100

ДЕШИФРОВАНИЕ БЛОКА 1:**1001111111001110**

- Начальное состояние:
L=10011111, R=11001110
- --- РАУНД 8 ---
- Ключ раунда:
0001010001100011
- $F(L, K) = F(10011111, 0001010001100011) = 11101110$
- $R \oplus F(L, K) = 11001110 \oplus 11101110 = 00100000$
- Новое состояние:
L=00100000, R=10011111
- --- РАУНД 7 ---
- Ключ раунда:
1000101000110001
- $F(L, K) = F(00100000, 1000101000110001) = 11011011$
- $R \oplus F(L, K) = 10011111 \oplus 11011011 = 01000100$
- Новое состояние:
L=01000100, R=00100000
- --- РАУНД 6 ---
- Ключ раунда:
1100010100011000
- $F(L, K) = F(01000100, 1100010100011000) = 10011001$